

Proyecto 1: Gramática de un lenguaje loco

Especificación del lenguaje : Quantum Coffee

Integrantes:

Andres Baldi – 2024088934

Tomás Blando – 2024800088

Justin Lacayo –

Abraham Solano – 2024132538

Dario Soto –

Tecnológico de Costa Rica

Curso: Compiladores e intérpretes

Profesor: Aurelio Sanabria Rodríguez

Fecha: 24/09/2026

Tabla de contenidos

1. Motivación	3
2. Descripción general del lenguaje	3
2.1 Temática e identidad	3
2.2 Convenciones y características	3
3. Análisis del lenguaje	4
3.1 Elementos sólidos	4
3.2 Elementos débiles	5
3.3 Elementos chistosos	6
3.4 Comparación con otros lenguajes	7
4. Gramática EBNF	8
4.1 Convenciones de la gramática	8
4.2 Gramática completa	8
5. Ejercicios en el lenguaje	10
5.1 Ejercicio 1 - Números y expresiones	10
5.2 Ejercicio 2 - Textos	10
5.3 Ejercicio 3 - Control de flujo	10
5.4 Ejercicio 4 - Listas	11
5.5 Ejercicio 5 - Integrador	11
5.6 Ejercicio 6 - Bonus cuántico	12
6. Memes	13
7. Conclusiones	16
8. Bibliografía	16

1. Motivación

Quantum Coffee nació mientras buscábamos un concepto gracioso para el lenguaje que pudiera ir más allá de simplemente cambiar las palabras reservadas. El café salió de forma bastante natural por ser algo muy presente en la cultura costarricense y en la vida diaria. Una de las primeras ideas que tuvimos fue que las variables tuvieran un tiempo de vida y que su estado fuera cambiando conforme no se utilizaban. Por ejemplo, una variable podía estar caliente y, si pasaba cierto tiempo sin usarse, se enfriaba.

A partir de esa idea de estados que podían cambiar, lo relacionamos con la computación cuántica. Nos pareció interesante mezclar algo tan cotidiano y tradicional como el café con algo que está casi en el polo opuesto, como la computación cuántica. De ahí salió la idea de que Quantum Coffee no solo tuviera la temática del café en sus palabras, sino también una parte cuántica que formara parte del lenguaje.

2. Descripción general del lenguaje

2.1 Temática e identidad

Quantum Coffee es un lenguaje que mezcla elementos de la cultura costarricense, principalmente el café, con conceptos de computación cuántica. La idea es que el lenguaje tenga dos partes. Por un lado está la parte clásica, que mantiene una forma de programar intuitiva para alguien que está empezando, parecida a Python o JavaScript. Por otro lado está la parte cuántica, que introduce conceptos básicos de computación cuántica de una forma sencilla.

Para la parte cuántica planteamos incorporar un simulador sencillo que permita trabajar con qubits y algunas operaciones básicas, como compuertas. No se busca que sea un sistema muy complejo, sino que sirva como un primer acercamiento a este tipo de programación.

Así, Quantum Coffee podría utilizarse como una introducción tanto a la programación clásica como a la programación cuántica, permitiendo que las personas que se interesen en el tema puedan avanzar después a herramientas más completas, como Qiskit o Gapi.

2.2 Convenciones y características

Quantum Coffee mantiene una estructura bastante parecida a la de otros lenguajes de programación, pero cambiando varias palabras reservadas para seguir con la temática del café. Para las variables globales utilizamos `cafetería`, mientras que las variables locales se pueden declarar directamente dentro de una función. Los tipos de datos principales son `grano` para números enteros, `poso` para números decimales, `granoMolido` para texto, `cafe` para valores booleanos y `taza` para listas. Las listas también mantienen un tipo definido, por ejemplo, una `taza grano` sería una lista que contiene valores de tipo `grano`.

Para las estructuras de control utilizamos `hierva` y `enfria` para las condiciones, mientras que para los ciclos tenemos `chorreando` y `recolar`. `chorreando` funciona de una forma parecida a un `while`, ya que repite un bloque mientras se cumpla una condición, y `recolar` permite recorrer los elementos de una lista. Las funciones se declaran utilizando `barista`, y para devolver un valor se utiliza `degustar`. El punto de entrada del programa también sigue esta misma idea y se declara como `barista chorreador()`.

También tenemos algunas funciones sencillas para la entrada y salida de información. `servir` se utiliza para mostrar valores y `leer` para obtener información. Los comentarios se escriben utilizando `#`, parecido

a Python, y se mantienen hasta el final de la línea. En general, intentamos que la sintaxis fuera sencilla de leer y que las palabras utilizadas tuvieran relación con la temática, pero sin cambiar demasiado la forma en que normalmente se estructura un programa.

3. Análisis del lenguaje

3.1 Elementos sólidos

Jerarquía de expresiones: La prioridad de los operadores se define directamente en la gramática siguiendo este orden:

expresion_or > expresion_and > expresion_not > expresion_relacional > expresion_aritmetica > termino > factor

En cada nivel se usan repeticiones { ... } en vez de recursión izquierda. Esto facilita convertir cada producción en una función dentro de un analizador descendente recursivo. Además, no hace falta utilizar una tabla externa para definir la prioridad de los operadores. Por ejemplo, en $2 + 3 * 4$, primero se resuelve $3 * 4$ y después se suma 2, porque $*$ se encuentra en un nivel más profundo que $+$.

Los bloques con llaves evitan el problema del "else colgante":

En lenguajes como C puede existir confusión sobre a cuál if pertenece un else cuando hay varias condiciones seguidas. En Quantum Coffee, hierve y enfria siempre utilizan bloques con llaves, por lo que es más fácil ver qué instrucciones pertenecen a cada condición.

```
hervir(a) {  
    hervir(b) { servir("a y b"); }  
}  
enfria {  
    servir("no a");  
}
```

La asignación como una sentencia:

El símbolo = solamente se utiliza al declarar una variable o al hacer una asignación. Dentro de una condición se utiliza ==. Esto evita errores comunes como escribir una asignación dentro de una condición por accidente.

```
hervir(temperatura = 90) { ... } # rechazado: "=" no pertenece a expresion
```

El alcance de las variables:

Las variables globales utilizan la palabra cafeteria, mientras que las variables locales no necesitan un prefijo. Gracias a esto, al leer el código se puede reconocer con mayor facilidad si una variable es global o local. También ayuda a evitar que se creen variables globales por accidente. Además, chorreador se declara utilizando barista, igual que las demás funciones, manteniendo una forma similar para declarar los subprogramas.

Tipos básicos y listas con un tipo definido:

El lenguaje incluye cadenas, booleanos, nulo y error como valores propios. También permite usar taza varias veces, por ejemplo `taza taza grano`, lo que permite representar estructuras como matrices. Como las listas mantienen un mismo tipo de datos y los parámetros indican el tipo que reciben, una declaración como `taza grano inventario` permite entender mejor qué clase de información espera una función.

Temática:

Las palabras reservadas siguen la temática de una cafetería. Por ejemplo, `barista` se utiliza para las funciones, `degustar` para retornar un valor, `servir` para mostrar una salida, `chorreando` y `recolar` para los ciclos, y `hierva` junto con `enfria` para las condiciones. Al mantener esta misma idea a lo largo del lenguaje, es más sencillo relacionar cada palabra con lo que hace una vez que se entiende la temática general.

3.2 Elementos débiles

Las funciones no dicen directamente qué tipo de dato devuelven: En el Ejercicio 5, por ejemplo, `auditarInventario` devuelve un café, pero eso no aparece en la firma:

```
barista auditarInventario(taza grano inventario) { ... degustar fuerte ; }
```

Entonces, para saber qué devuelve la función, toca revisar el cuerpo y ver qué aparece en los `degustar`. Esto también hace que el analizador semántico tenga que averiguarlo por su cuenta. Además, tampoco existe una forma de indicar claramente que una función no devuelve nada.

Las listas se quedan un poco cortas en operaciones: Por ahora solo permiten insertar, extraer y buscar. No se puede entrar directamente a una posición ni preguntar cuántos elementos tiene la lista. Por ejemplo:

```
granoMolido primero = menu[0] ; # no existe: no se puede acceder por posición
grano n = menu.largo() ; # no existe: no hay una operación para obtener el tamaño
```

Si se quiere ver un elemento sin sacarlo de la lista, básicamente hay que hacerlo dentro de `recolar`. Tampoco se puede cambiar directamente lo que está guardado en una posición. Y `buscar` solamente responde si el elemento está o no, pero no dice en qué posición aparece.

Los números también tienen algunas limitaciones: `numero_entero` está formado solo por dígitos y `factor` no acepta un `-` directamente delante de un número. Entonces algo como `grano deuda = -5` ; no funciona y habría que escribir `0 - 5`. Tampoco se pueden escribir valores como `.5` ni usar notación científica, así que por ese lado el formato de números es bastante básico.

El control de flujo es bastante sencillo: No existe algo como `else if`, así que cuando hay varias condiciones toca ir metiendo un `hierva` dentro del `enfria` anterior:

```
hierva (temperatura < 50) {
    servir("Frio") ;
} enfria {
    hierva (temperatura < 90) {
        servir("Tibio") ;
    }
}
```

```

    } enfria {
        servir("Listo");
    }
}

```

También hacen falta algunas instrucciones comunes como for con contador, break y continue. Si se quiere repetir algo una cantidad específica de veces, hay que crear una variable auxiliar y resolverlo usando chorreando, como pasa en el Ejercicio 3. Funciona, pero claramente se vuelve un poco más largo de lo necesario.

Las cadenas y la salida: `caracter_cadena` no permite usar comillas dobles dentro del texto ni saltos de línea, y tampoco hay secuencias de escape. Entonces algo como `servir("Dijo \"hola\"")`; no sería válido. Además, `servir` recibe una sola expresión, por lo que si se quieren mostrar varias cosas hay que unir las usando `+`. Tampoco queda definido qué debería pasar al mezclar tipos distintos, por ejemplo un `granoMolido` con un `poso`. Otro detalle curioso es que los identificadores solo aceptan letras ASCII, así que no se podrían usar ñ ni tildes, lo cual se siente un poco raro para un lenguaje que está pensado en español.

3.3 Elementos chistosos

El humor del lenguaje viene de que cada construcción tiene una lectura literal de cafetería, y se usa como chiste al relacionarlo con elementos del lenguaje

Los tipos tienen la lógica del grano: `grano` es un entero porque el grano se cuenta de uno en uno, y `taza` es un arreglo porque es lo que contiene el café. `descafeinado` es el nulo ideal: sigue siendo café, pero sin la parte más vital, igual que un `null` es un valor sin contenido.

El humor es local: `chorreador` es el colador tradicional de café costarricense, y con eso el punto de entrada es literalmente el aparato por donde pasa todo el programa, como el agua hirviendo para chorrear café. El `cafe` en sí es el booleano y puede estar de 2 maneras `ralo` es como se le dice al café aguado, y por eso es el `false` y `true` es `fuerte` su contrario. `SeQuemoElCafe` es el error, y lo gracioso es que, como en la vida real, una vez quemado no hay forma de deshacerlo: es un valor que la gramática no permite capturar (ver 3.2).

El programa se lee como una escena de cafetería, o como la preparación de una taza: El Ejercicio 3 realiza un ciclo `chorreando` (`temperatura < 90`) hasta que el agua llega al punto y luego pregunta con `hierve`. El Ejercicio 5 escribe `hierve` (`falta_cafe == fuerte` `ConAzucar` `total_granos < 50`) y responde con "Alerta crítica: Hay que tostar mas cafe urgente". Un condicional sobre inventarios se ve como una emergencia de cafetería.

El único intruso es `leer`: Es la única palabra reservada fuera de la metáfora.

3.4 Comparación con otros lenguajes

Tabla 1 – Decisiones puntuales frente a C y Java

Aspecto	C	Java	Quantum Coffee
Punto de entrada	<code>int main()</code> en cualquier posición	<code>main</code> dentro de una clase	<code>barista chorreador()</code> , sin parámetros ni retorno, siempre al final
Tipo de retorno	Antes del nombre	Antes del nombre	No se declara
Booleanos	<code>int</code> (o <code>stdbool.h</code>)	<code>boolean</code>	<code>cafe</code> con <code>fuerte/ralo</code>
Nulo y errores	<code>NULL</code> , <code>errno</code>	<code>null</code> , excepciones	<code>descafeinado</code> , <code>SeQuemoElCafe</code> (valores sin captura)
Asignación en condición	Permitida (<code>if (x = 5)</code>)	Solo con <code>boolean</code>	Imposible (es sentencia)
Else-if	<code>else if</code>	<code>else if</code>	No existe, se anida en <code>enfria</code>
Recorrer una lista	<code>for</code> con índice	<code>for (T x : lista)</code>	<code>recolar (T x en lista)</code>
Variables globales	Nivel de archivo, sin marca	<code>static</code> en una clase	<code>cafeteria</code>

Listas y ciclos: `taza` se comporta como el `ArrayList` de Java, donde `insertar` corresponde a `add`, `extraer(i)` a `remove(i)` y `buscar` a `contains`. Pero no tiene `get` y `size`, que son lo que hace usable a `ArrayList`. `recolar` mezcla dos estilos: la palabra `in` de Python (`for x in lista`) con el tipo explícito de Java (`for (String x : lista)`).

Nulo y error como valores: `descafeinado` se parece al `null` de Java, y si puede asignarse a cualquier tipo (la especificación no lo aclara); además, tiene los mismos problemas de referencias nulas. `SeQuemoElCafe` se acerca al tipo `error` de Go, que también es un valor, pero aquí no hay retorno múltiple ni forma de comprobarlo, que es lo que hace útil el patrón en Go.

Ensamblador: El ciclo del Ejercicio 3 se reduce a una comparación y dos saltos:

```

    mov  eax, 20      ; temperatura = 20
ciclo: cmp  eax, 90    ; temperatura < 90 ?
      jge  fin
      add  eax, 10     ; temperatura = temperatura + 10
      jmp  ciclo
fin:

```

En ASM no existen bloques, tipos ni precedencia. Todo eso lo aporta el lenguaje, y por eso la gramática debe ser más precisa en un lenguaje de alto nivel.

4. Gramática EBNF

4.1 Convenciones de la gramática

La gramática usa `::=` para las asignaciones. Las llaves `{ ... }` son repeticiones de cero o más elementos, los corchetes `[ ... ]` significan elementos opcionales y las comillas dobles señalan símbolos terminales. Los comentarios son ignorados del lenguaje y no necesitan aparecer dentro de cada producción sintáctica. Las palabras reservadas no pueden usarse como identificadores.

4.2 Gramática completa

```
(* 1. Tipos de datos *)
tipo ::= "grano"           (* int *)
      | "poso"            (* float *)
      | "granoMolido"     (* string *)
      | "taza" tipo       (* array de un tipo *)
      | "cafe"            (* boolean *)

(* 2. Estructura del programa *)
programa ::= { declaracion_global | declaracion_funcion }
declaracion_chorreador
declaracion_global ::= "cafeteria" tipo identificador [ "=" expresion ] ";"
declaracion_chorreador ::= "barista" "chorreador" "(" ")" bloque

(* 3. Funciones *)
declaracion_funcion ::= "barista" identificador "(" [ parametros ] ")"
bloque
parametros ::= parametro { "," parametro }
parametro ::= tipo identificador
llamada_funcion ::= identificador "(" [ argumentos ] ")"
argumentos ::= expresion { "," expresion }

(* 4. Bloques y sentencias *)
bloque ::= "{" { sentencia } "}"

sentencia ::= declaracion_var_local
            | declaracion_qubit
            | asignacion
            | estructura_condicional
            | bucle_chorreando
            | bucle_recolar
            | llamada_leer
              | llamada_servir
              | llamada_funcion ";"
              | operacion_lista ";"
              | operacion_cuantica ";"
              | medicion_cuantica ";"
              | retorno_degustar

declaracion_var_local ::= tipo identificador [ "=" expresion ] ";"
asignacion ::= identificador "=" expresion ";"
llamada_servir ::= "servir" "(" expresion ")" ";"
llamada_leer ::= identificador "=" "leer" "(" ")" ";"
retorno_degustar ::= "degustar" expresion ";"

(* 5. Estructuras de control *)
```

```

estructura_condicional ::= "hierve" "(" expresion ")" bloque [ "enfria" bloque
]
bucle_chorreando      ::= "chorreando" "(" expresion ")" bloque
bucle_recolar         ::= "recolar" "(" tipo identificador "en" identificador
")" bloque

(* 6. Operadores lógicos *)
op_or                  ::= "SinAzucar"
op_and                 ::= "ConAzucar"
op_not                 ::= "Amargo"
op_relacional          ::= "==" | "!=" | "<" | "<=" | ">" | ">="

(* 7. Jerarquía de expresiones *)
expresion              ::= expresion_or
expresion_or           ::= expresion_and { op_or expresion_and }
expresion_and          ::= expresion_not { op_and expresion_not }
expresion_not          ::= [ op_not ] expresion_relacional
expresion_relacional   ::= expresion_aritmetica [ op_relacional
expresion_aritmetica ]
expresion_aritmetica   ::= termino { ( "+" | "-" ) termino }
termino                ::= factor { ( "*" | "/" | "%" ) factor }

factor                 ::= literal
                        | identificador
                        | llamada_funcion
                        | operacion_lista
                        | medicion_cuantica
                        | "(" expresion ")"

operacion_lista        ::= identificador "." ( "insertar" "(" expresion ")"
                        | "extraer" "(" expresion ")"
                        | "buscar" "(" expresion ")" )

(* 8. Literales y terminales *)
literal                ::= numero_entero
                        | numero_decimal
                        | cadena
                        | "fuerte"
                        | "ralo"
                        | "descafeinado"
                        | "SeQuemoElCafe"
                        | arreglo

arreglo                ::= "[" [ expresion { "," expresion } ] "]"

identificador          ::= letra { letra | digito | "_" }
numero_entero          ::= digito { digito }
numero_decimal         ::= digito { digito } "." digito { digito }
cadena                 ::= '"' { caracter_cadena } '"'

letra                  ::= "A".. "Z" | "a".. "z"
digito                 ::= "0".. "9"
caracter_cadena        ::= ? cualquier caracter excepto comillas dobles y salto
de linea ?
comentario              ::= "#" { ? cualquier caracter excepto salto de linea ?
}

(* 9. Extensión cuántica *)
declaracion_qubit      ::= "qubit" identificador ";"
operacion_cuantica     ::= compuerta_unaria
                        | compuerta_binaria

```

```

compuerta_unaria      ::= ( "H" | "X" | "Z" ) "(" identificador ")"
compuerta_binaria     ::= "CNOT" "(" identificador "," identificador ")"
medicion_cuantica     ::= "medir" "(" identificador ")"

```

5. Ejercicios en el lenguaje

Los siguientes ejercicios demuestran el uso de números, textos, estructuras de control, listas y composición de funciones en Quantum Coffee. En conjunto cubren las construcciones principales definidas por la gramática EBNF.

5.1 Ejercicio 1 - Números y expresiones

Descripción: Demuestra declaraciones locales con los tipos grano y peso, operaciones aritméticas y salida mediante servir.

```

barista chorreador() {
    grano sacos_bodega = 15 ;
    peso peso_por_saco = 45.5 ;
    peso peso_total = sacos_bodega * peso_por_saco ;

    servir(peso_total) ;
}

```

Reglas EBNF utilizadas: declaracion_chorreador, bloque, declaracion_var_local, expresion_aritmetica, termino, numero_entero, numero_decimal y llamada_servir.

5.2 Ejercicio 2 - Textos

Descripción: Demuestra el manejo de cadenas con granoMolido, asignación de literales de texto, concatenación mediante + y salida con servir.

```

barista chorreador() {
    granoMolido cliente = "Don Juan" ;
    granoMolido pedido = "Capuchino doble" ;

    servir("Orden lista para: " + cliente) ;
    servir("Bebida: " + pedido) ;
}

```

Reglas EBNF utilizadas: declaracion_chorreador, bloque, declaracion_var_local, cadena, expresion_aritmetica, factor y llamada_servir.

5.3 Ejercicio 3 - Control de flujo

Descripción: Demuestra repetición con chorreando, actualización de variables y bifurcación con hierva/enfria a partir de expresiones relacionales.

```

# Uso de repeticion (chorreando) y bifurcacion (hierva/enfria)
barista chorreador() {
    grano temperatura = 20 ;

```

```

    chorreando (temperatura < 90) {
        temperatura = temperatura + 10 ;
    }

    hierve (temperatura >= 90) {
        servir("Agua al punto ideal para chorrear") ;
    } enfria {
        servir("Falta fuego, el agua esta fria") ;
    }
}

```

Reglas EBNF utilizadas: declaracion_chorreador, declaracion_var_local, bucle_chorreando, estructura_condicional, asignacion, expresion_relacional, expresion_aritmetica y llamada_servir.

5.4 Ejercicio 4 - Listas

Descripción: Demuestra listas tipadas con taza, literales de arreglo, las operaciones insertar, buscar y extraer, y el recorrido de elementos mediante recolar.

```

# Manejo de arreglos e invocacion de metodos nativos
barista chorreador() {
    taza granoMolido menu = ["Espresso", "Americano"] ;

    menu.insertar("Latte") ;
    cafe hay_latte = menu.buscar("Latte") ;
    granoMolido servido = menu.extraer(0) ;

    servir("Bebida servida al cliente:") ;
    servir(servido) ;

    servir("Restante en el menu:") ;
    recolar (granoMolido bebida en menu) {
        servir(bebida) ;
    }
}

```

Reglas EBNF utilizadas: tipo, declaracion_chorreador, declaracion_var_local, arreglo, operacion_lista, bucle_recolar, cadena, numero_entero y llamada_servir.

5.5 Ejercicio 5 - Integrador

Descripción: Integra funciones, parámetros, listas, booleanos, ciclos, condicionales, operadores lógicos y retorno con degustar. La identidad de Quantum Coffee se refleja en palabras reservadas temáticas como barista, recolar, hierve, ConAzucar y degustar.

```

# Uniendo tipos, funciones, booleanos, ciclos y condicionales complejos
barista auditarInventario(taza grano inventario) {
    grano total_granos = 0 ;
    cafe falta_cafe = ralo ;

    recolar (grano cantidad en inventario) {
        hierve (cantidad <= 5) {
            falta_cafe = fuerte ;
        }
        total_granos = total_granos + cantidad ;
    }
}

```

```

    hierva (falta_cafe == fuerte ConAzucar total_granos < 50) {
        servir("Alerta critica: Hay que tostar mas cafe urgente") ;
        degustar ralo ;
    } enfria {
        servir("Inventario de la cafeteria saludable") ;
        degustar fuerte ;
    }
}

barista chorreador() {
    taza grano bodega = [10, 4, 20, 15] ;
    cafe estado_tienda = auditarInventario(bodega) ;
}

```

Reglas EBNF utilizadas: declaracion_funcion, parametros, parametro, tipo, bucle_recolar, estructura_condicional, expresion_and, op_and, expresion_relacional, retorno_degustar, llamada_funcion, declaracion_chorreador y arreglo.

5.6 Ejercicio 6 - Bonus cuántico

Descripción: Introduce la extensión cuántica del lenguaje mediante declaración de qubits, compuertas H, X, Z y CNOT, y medición de resultados como valores cafe.

```

# Bonus cuantico: compuertas y medicion
barista chorreador() {
    qubit q0 ;
    qubit q1 ;

    X(q0) ;
    H(q0) ;
    Z(q0) ;
    CNOT(q0, q1) ;

    cafe resultado0 = medir(q0) ;
    cafe resultado1 = medir(q1) ;

    servir(resultado0) ;
    servir(resultado1) ;
}

```

Reglas EBNF utilizadas: declaracion_chorreador, bloque, declaracion_qubit, operacion_cuantica, compuerta_unaria, compuerta_binaria, medicion_cuantica, declaracion_var_local y llamada_servir.

6. Memes

Figura 1 – Coffee

Lenguajes de Programacion normales	True  False 
Quantum Coffee	Maybe 

Figura 2 – Gato de Schrodinger?

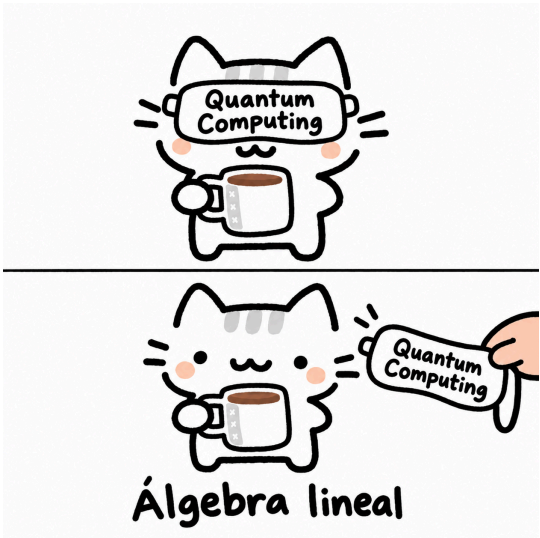


Figura 3 – Levels



Figura 4 – Aura

En 10 minutos llego, estoy
haciendo cafe



Figura 5 – No words...

como veo a mi compa despues de escribir
en quantum coffee



7. Conclusiones

Ya tenemos definida la identidad de Quantum Coffee. La temática del café está presente en diferentes partes del lenguaje, como sus tipos de datos y sus funciones, y no solo en el nombre. Además, las ideas previas que tuvimos nos ayudaron a llegar a lo que estamos desarrollando.

Logramos definir una gramática que nos permite trabajar con estructuras básicas del lenguaje. La parte cuántica es lo que nos termina diferenciando, ya que usa qubits, compuertas y algunos conceptos básicos de computación cuántica. Va a ser complejo, sí, pero sería un excelente proyecto.

Así, Quantum Coffee no es solo algo cercano para nosotros. La programación es sencilla, como lo que podemos encontrar en Python, y la parte cuántica le da un giro diferente, una nueva dificultad y un nuevo horizonte para trabajar.

8. Bibliografía

IBM Quantum. (s. f.). Basics of quantum information.

<https://quantum.cloud.ibm.com/learning/en/courses/basics-of-quantum-information>

IBM Quantum. (s. f.). Quantum circuit model. <https://quantum.cloud.ibm.com/docs/en/api/qiskit/circuit>

Python Software Foundation. (s. f.). Referencia del lenguaje Python.

<https://docs.python.org/es/3/reference/>

Quantinuum. (s. f.). Functions. Guppy Documentation.

https://docs.quantinuum.com/guppy/language_guide/functions.html

Quantinuum. (s. f.). Guppy Language Guide.

https://docs.quantinuum.com/guppy/language_guide/language_guide_index.html

Quantinuum. (2025, 20 de agosto). Guppy: Programming the Next Generation of Quantum Computers.

<https://www.quantinuum.com/blog/guppy-programming-the-next-generation-of-quantum-computers>

Quantinuum. (s. f.). guppylang [Código fuente]. GitHub. <https://github.com/Quantinuum/guppylang>

van Rossum, G., Warsaw, B., & Coghlan, A. (2001). PEP 8 – Style Guide for Python Code. Python Software Foundation. <https://peps.python.org/pep-0008/>